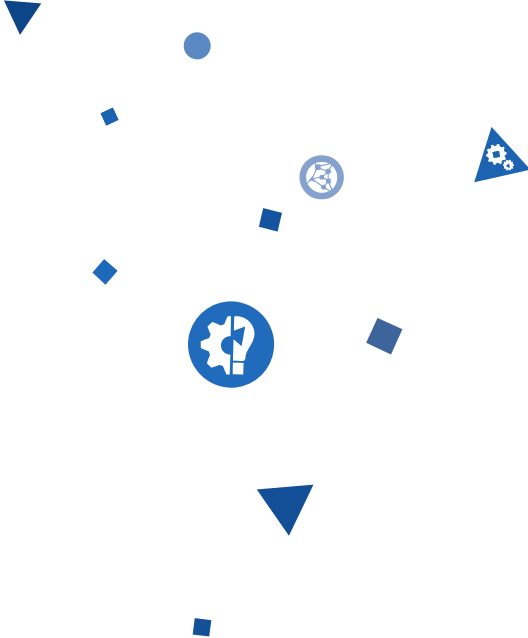


A collection of blue geometric shapes including triangles, squares, and circles, some containing icons like a gear and a question mark, arranged in a loose cluster on the left side of the slide.

From Bytecode to Native Code in HotSpot JVM

Ionuț Baloșin

Software Architect

ibalosin@luxoft.com

 [@ibalosin](https://twitter.com/ibalosin)

About Me

- **Software Architect** @ www.luxoft.com
 - 9 years of software development experience
- **Technical Trainer** @ www.luxoft-training.com
 - *"Java Performance and Tuning"* course
 - *"Software Architecture Methodology"* course

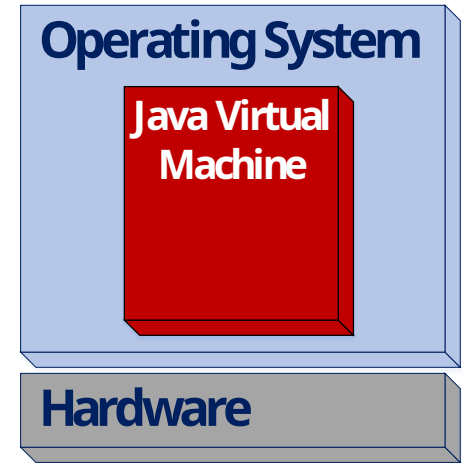
Agenda

- Understanding Bytecode
- How to Disassemble Bytecode
- Bytecode Execution Workflow
 - Interpreter
 - JIT Compiler
- How to see Native Code from Bytecode
- Demo
- Micro-benchmarking advice

HotSpot Java Virtual Machine

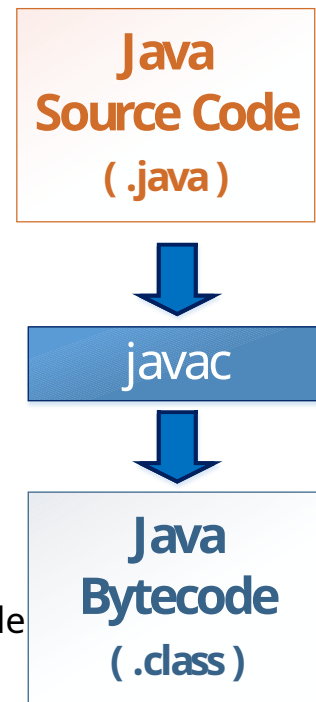
➤ JVM

- process virtual machine for application code
- built-in memory management system
- provides robust and secure execution
- stack-based implementation



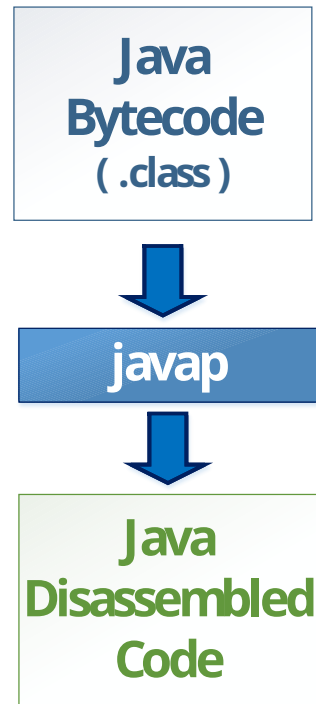
javac

- Bytecode is generated using **javac** based on Java source code
- There are about 200 bytecode instructions (**opcodes**) in use
 - each **opcode** is represented by a single byte → **bytecode**
 - **opcodes** are organized in families
- Bytecode does not have direct access to any memory location
- Bytecode does not contain some high-level features present in the Java Source Code
 - E.g. comments, for, while, etc

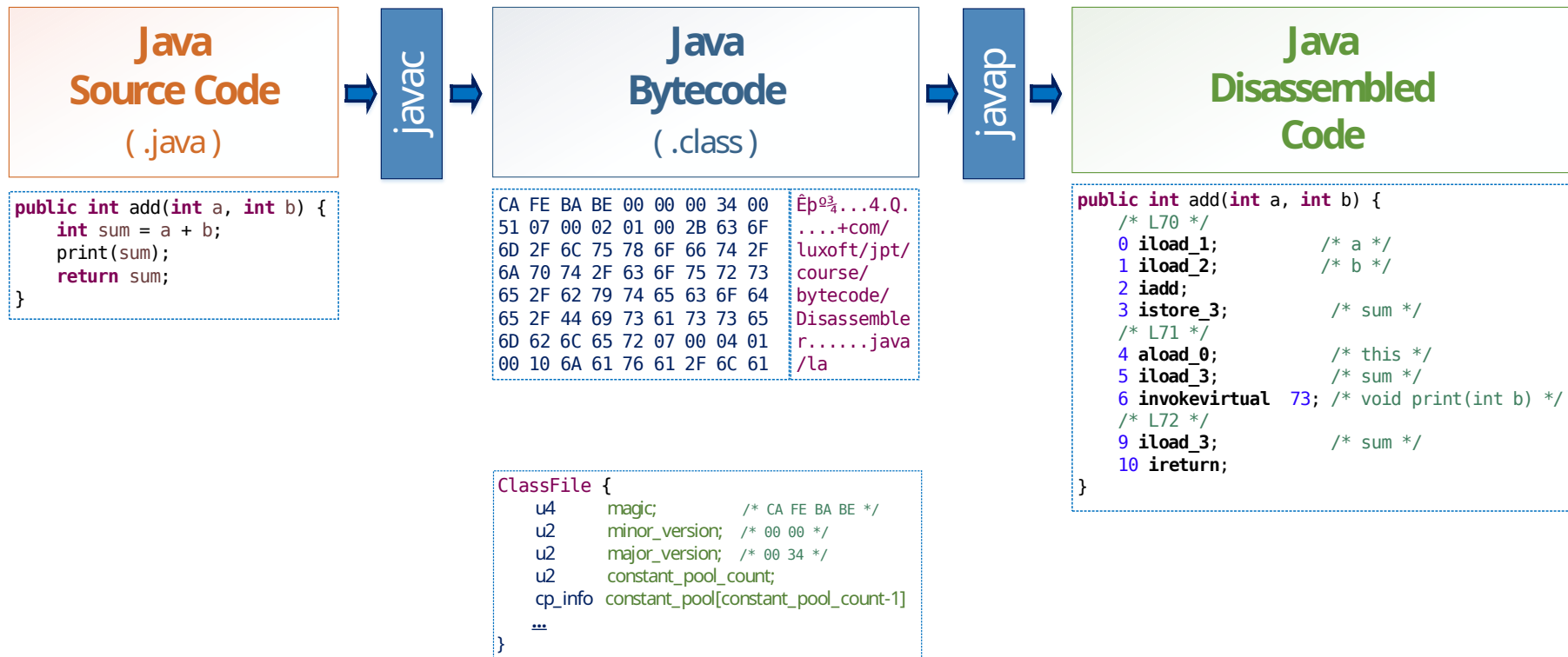


javap

- Disassembled code is generated using **javap** based on generated Bytecode
- **javap** - Java Class File Disassembler
 - by default it shows public and default methods
 - `javap -c <path_to_class>`
- IDE Disassemblers:
 - Eclipse: Bytecode Visualizer
 - IntelliJ IDEA: ASM Bytecode Outline



From Source Code to Disassembled Code



OpCode Families

```
public int add(int a, int b) {  
    /* L70 */  
    0 iload_1;          /* a */  
    1 iload_2;          /* b */  
    2 iadd;  
    3 istore_3;         /* sum */  
    /* L71 */  
    4 aload_0;          /* this */  
    5 iload_3;          /* sum */  
    6 invokevirtual 73; /* void print(int b) */  
    /* L72 */  
    9 iload_3;          /* sum */  
    10 ireturn;  
}
```

ARITHMETIC OPCODES

add	add two values from top of stack and store the result
sub	subtracts two values from top of stack and store the result
div	divide two values from top of stack and store the result
mult	multiplies two values from top of stack and store the result

INVOCATION OPCODES

invokeinterface	calls an interface method
invokespecial	calls a non-overridable instance method
invokevirtual	calls a regular overridable instance method
invokestatic	calls a static method
invokedynamic	dynamic invocation

LOAD & STORE OPCODES

load	load a value from Local Variable Array into the stack
ldc	load a constant from the Constant Pool into the stack
store	store a value into Local Variable Array and remove it from stack
dup	duplicate the value on top of the stack

Local Variable Array

```
public int add(int a, int b) {  
    int sum = a + b;  
    print(sum);  
    return sum;  
}
```



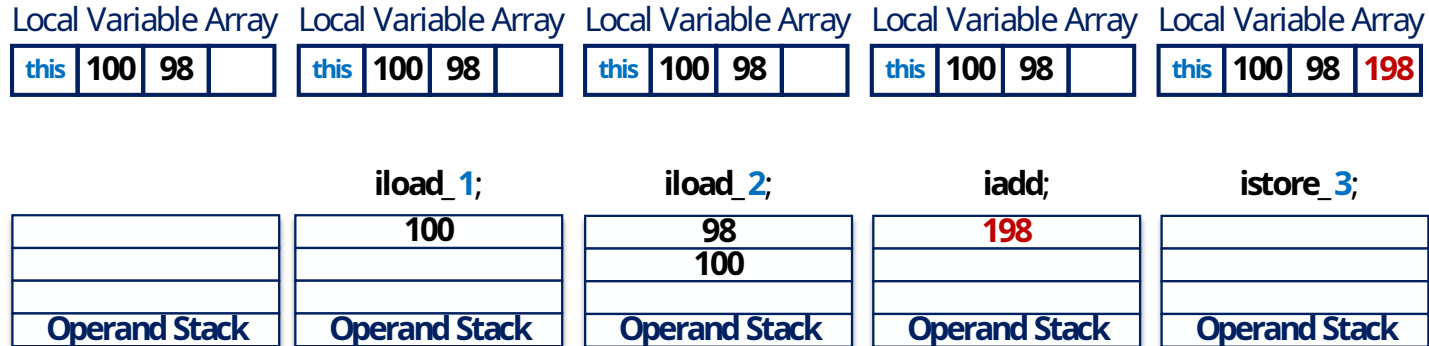
```
public int add(int a, int b) {  
    /* L70 */  
    0 iload_1;           /* a */  
    1 iload_2;           /* b */  
    2 iadd;  
    3 istore_3;          /* sum */  
    /* L71 */  
    4 aload_0;           /* this */  
    5 iload_3;           /* sum */  
    6 invokevirtual 73;  /* void print(int sum) */  
    /* L72 */  
    9 iload_3;           /* sum */  
    10 ireturn;  
}
```

Local Variable Array
index

How Operand Stack and Local Variable Array work

```
public int add(int a, int b) {  
    /* ... */  
    0 iload_1;           /* a */  
    1 iload_2;           /* b */  
    2 iadd;  
    3 istore_3;          /* sum */  
    /* ... */  
}
```

Stack Frame



Runtime Constant Pool

```
public int add(int a, int b) {  
    int sum = a + b;  
    print(sum);  
    return sum;  
}
```



```
public int add(int a, int b) {  
    /* L70 */  
    0 iload_1;          /* a */  
    1 iload_2;          /* b */  
    2 iadd;             /* sum */  
    3 istore_3;         /* this */  
    /* L71 */          /* sum */  
    4 aload_0;          /* void print(int sum) */  
    5 iload_3;          /* sum */  
    6 invokevirtual 73;  
    /* L72 */          /* sum */  
    9 iload_3;  
    10 ireturn;  
}
```

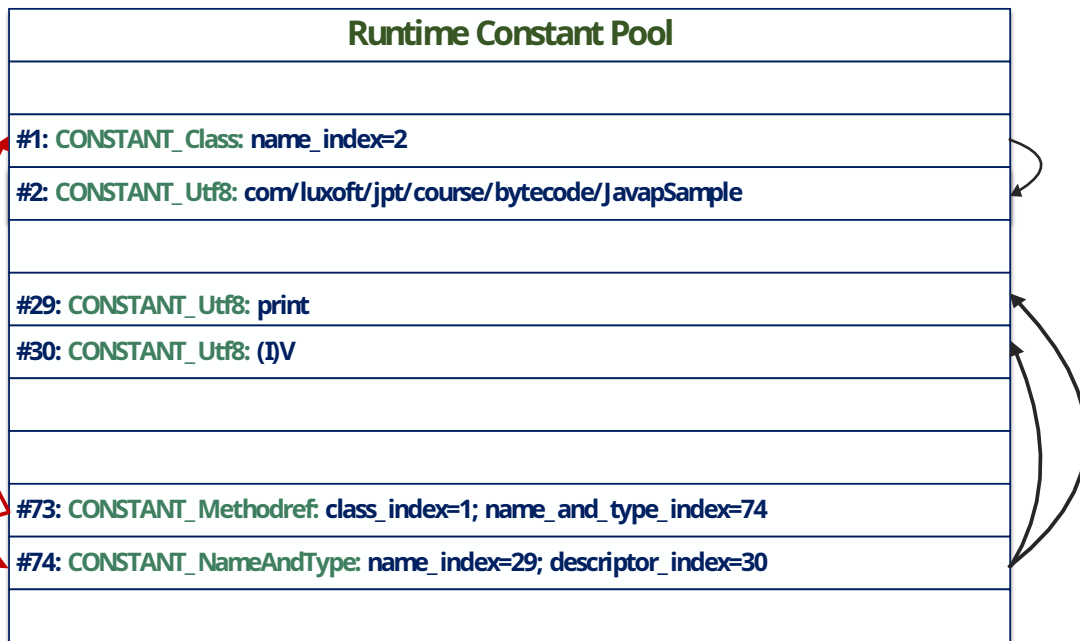
Local Variable Array
index

Runtime Constant Pool
reference

How Runtime Constant Pool works

- Ordered set of constants used by the type
- Entries have usually 2 bytes, referenced by index
- Plays a central role in the dynamic linking of Java

```
public int add(int a, int b) {  
    /* ... */  
    6 invokevirtual 73;      /* void print(int sum) */  
    /* ... */  
}
```



Exceptions handling at Bytecode level

```
public static int remainder(int a, int b) throws DivideByZeroException {  
    try {  
        return a % b;  
    } catch (ArithmeticException e) {  
        throw new DivideByZeroException();  
    }  
}
```

```
public static int remainder(int a, int b) throws com.luxoft.jpt.course.bytecode.DivideByZeroException {  
    /* L26 */  
    0 iload_0;          /* a */  
    1 iload_1;          /* b */  
    2 irem;  
    3 ireturn;  
    /* L27 */  
    4 astore_2;         /* e */  
    /* L28 */  
    5 new 17;  
    8 dup;  
    9 invokespecial 19; /* com.luxoft.jpt.course.bytecode.DivideByZeroException() */  
    12 athrow;  
  
    /*  
    ExceptionTable  
    */  
    /* -----+-----+-----+-----+ */  
    /* start_pc | end_pc | handler_pc | catch_type */  
    /* -----+-----+-----+-----+ */  
    /*      0 |      3 |      4 | java.lang.ArithmeticException */  
    /* -----+-----+-----+-----+ */  
}
```

Quizz

- Spot few ambiguities based on below decompiled class:

```
public class MyDouble implements MyDoubleInterface<Integer> {  
  
    @Override  
    public Integer getDouble() {  
        return Integer.valueOf(10) * 2;  
    }  
}
```



```
public class MyDouble implements com.luxoft.jpt.course.bytecode.MyDoubleInterface<java.lang.Integer> {  
  
    /* compiled from MyDouble.java */  
  
    public MyDouble() {...}  
  
    public java.lang.Integer getDouble() {...}  
  
    public volatile java.lang.Object getDouble() {...}  
}
```

Quizz

- ❑ Where method **access\$0** comes from?

```
public class JavapOuterClass {  
  
    private void foo() {  
    }  
  
    public class MyInnerClass {  
        public void run() {  
            foo();  
        }  
    }  
}
```



```
public class JavapOuterClass {  
  
    /* compiled from JavapOuterClass.class */  
  
    public JavapOuterClass() {...}  
  
    private void foo() {...}  
  
    static void access$0(com.luxoft.jpt.course.bytecode.JavapOuterClass arg0) {  
        arg0.foo();  
    }  
}
```

Quizz

- ❑ Where method `$0` comes from? How it could be **private** and **static** inside the Interface?

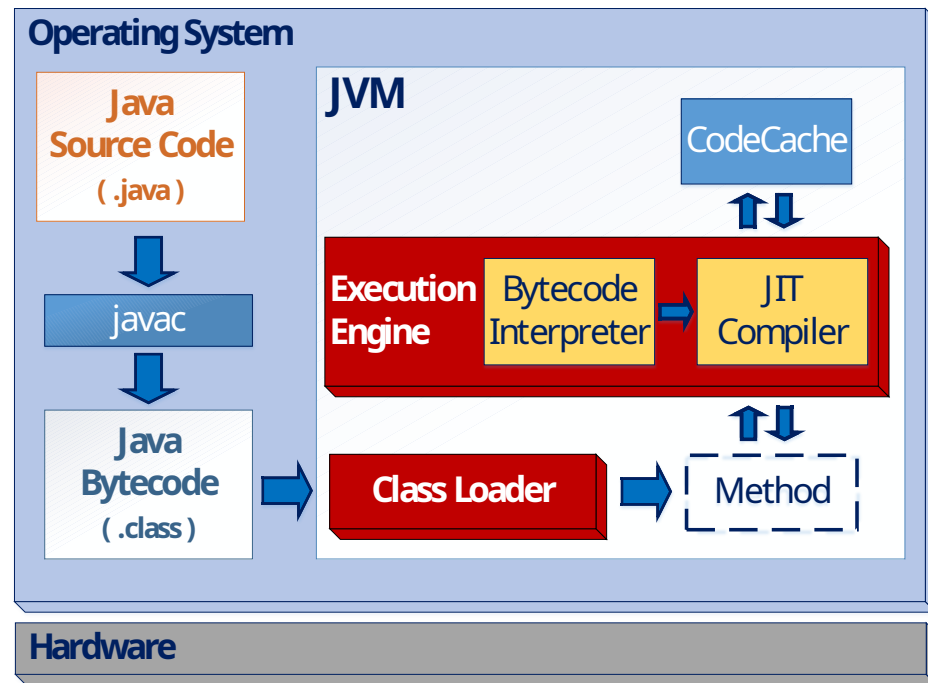
```
public interface MyPrinter {  
  
    default void print() {  
        Runnable r = () -> { System.out.println("Hello Voxxed Bucharest"); };  
        r.run();  
    }  
}
```



```
public interface MyPrinter {  
  
    /* compiled from MyPrinter.class */  
  
    default void print() {  
        invokedynamic [ LambdaMetafactory.metafactory,  
                        MethodHandle(Runnable.run),  
                        MethodHandle(lambda$0) ] ()  
    }  
  
    private static void lambda$0() {  
        invokevirtual java.lang.System.out.println("Hello Voxxed Bucharest")  
    }  
}
```


Execution Engine

- **HotSpot** rationales
 - Weak Generation Hypothesis
 - **memory management and Garbage Collector**
 - use runtime to take best optimization decisions
 - **Execution Engine**
- **Execution Engine**
 - Interpreter
 - Just-In-Time Compiler



HotSpot Interpreter

- When the JVM first starts up, thousands of methods are called
 - compiling all methods can significantly affect startup time
 - "*~90% of time is spent in ~2% of methods*"
- Solution: methods are not compiled first time they are called, they are **interpreted**
 - HotSpot JVM starts running in **Interpreter** mode
- Each method have a call count, once it reaches an **invocation threshold** it gets compiled
- hence **hot spots** (HotSpot JVM)

JVM argument	Explanations
-XX:CompileThreshold	Number of method invocations/branches before compiling [client 1.5k invocations; server 10k invocations]

HotSpot Just-In-Time Compiler

➤ Client (-client) C1

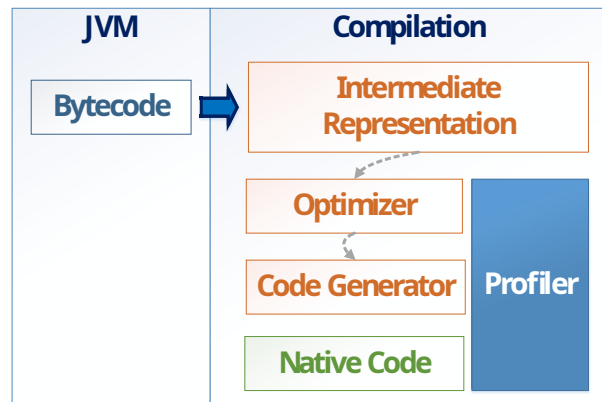
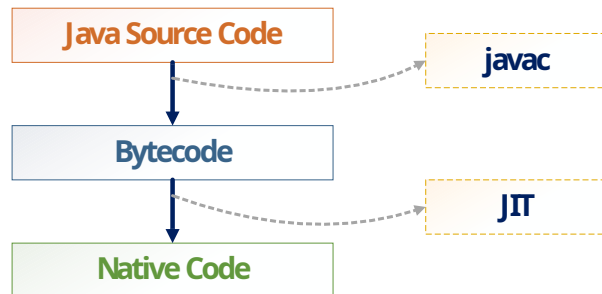
- fast code generation of acceptable quality
- rock-solid and proved optimizations
- doesn't need a **Profiler**
- compilation threshold: **1,5k** invocations

➤ Server (-server) C2

- highly optimized code
- many aggressive and speculative optimizations
- does need a **Profiler**
- compilation threshold: **10k** invocations

➤ Tiered Mode (-XX:+TieredCompilation) C1 + C2

- Level 0 = Interpreter
- Level 1-3 = **C1**
 - **C1** without profiling
 - **C1** with basic profiling
 - **C1** with full profiling
- Level 4 = **C2**



Welcome to HotSpot Optimizations' World

optimistic nullness assertions
graph-coloring register allocation
instruction peepholing escape analysis
address mode matching devirtualization
algebraic simplification inlining graph integration autobox elimination
de-reflection
array length strengthening expression hoisting memory value tracking
exact type inference
optimistic N-morphic inlining integer range typing
reassociation card-mark elimination null check elimination
switch balancing loop vectorization heat-based code layout
local code scheduling iteration range splitting constant folding
copy removal merge-point splitting delay slot filling
expression sinking loop peeling constant splitting
global code motion loop unrolling copy coalescing
throw inlining safepoint elimination untaken branch pruning
local code bundling range check elimination operator strength reduction
adjacent store fusion on-stack replacement
branch frequency prediction lock fusion
call frequency prediction
redundant store elimination
memory value inference live range splitting lock elision
type test elimination DFA-based register allocation
dead code elimination linear scan register allocation
type test strength reduction
optimistic type assertions

Just-In-Time Compiler optimizations

➤ Dynamic de-virtualization

- if JIT only sees 1 implementation of a virtualized method
 - it becomes a mono-morphic call

➤ Dynamic de-optimization

- if JIT discovers another implementation it must *de-optimize* and might *re-optimize* for second implementation
 - it becomes a bi-morphic call

➤ Ininsics

- optimizations done specifically to CPU
- do not inline, but inserts “best” native code
- Examples: `CAS`, `String::equals`; `Math::*`; `System::arrayCopy`; `Object::hashCode`

Method inlining

```
double getInterestRate() {  
    return (canComputeInterestRate()) ? 0 : (3 * amount / 1200);  
}  
  
boolean canComputeInterestRate() {  
    return amount != 0;  
}
```



```
double getInterestRate_() {  
    return (amount != 0) ? 0 : (3 * amount / 1200);  
}
```

HotSpot JVM arguments

JVM argument	Explanations
-Djava.compiler=none	Disables JIT Compiler
-client	Activates client (C1) compilation
-server	Activates server (C2) compilation
-XX:+TieredCompilation	Activates tiered (C1+C2) compilation
-Xbatch	Switches from background to foreground JIT compilation task until completed. It gets more deterministic behavior
-XX:+PrintCompilation	Prints time spent in JIT Compiler
-XX:+PrintInlining	Prints what methods get inlined
-XX:-CITime	Prints time spent in JIT Compiler.
-XX:MaxInlineSize	Specifies maximum number of bytecode instructions in a method which gets inlined. It does not to look at the invocation count [default 35bytes]

How to see Native Code from Bytecode

➤ HSDIS

➤ <https://wiki.openjdk.java.net/display/HotSpot/PrintAssembly>

JVM argument	Description
<code>-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly</code>	Activates for assembly printing

Demo Time

- Direct vs. Getter/Setter fields access members
 - interpreted mode
 - mixed compiled mode
- Mono-morphic / bi-morphic / mega-morphic virtual method calls
- JIT Watch

JITWatch - bi-morphic call

TriView - Source, Bytecode, Assembly Viewer - JITWatch

Class: Member:

☒ Source ☒ Bytecode ☒ Assembly ☐ Chain ☐ Journal ☒ Mouse Follow

Bytecode size: Native size: Compile time (...):

Source	Bytecode (double click for JVM spec)	Assembly ☐ Local labels
27 // Run this with -XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly and 28 doTest(Square.class.getSimpleName(), square, ITERATIONS); 29 System.out.printf("Running application with Java version [%s] \n", System. 30 31 Shape square = new Square(25,33); 32 Shape rectangle = new Rectangle(20.75, 30.25); 33 Shape rightTriangle = new RightTriangle(20.50, 30.25); 34 35 System.out.printf("Starting warm-up ... \n"); 36 doTest(Square.class.getSimpleName(), square, ITERATIONS); 37 doTest(Rectangle.class.getSimpleName(), rectangle, ITERATIONS); 38 //doTest(RightTriangle.class.getSimpleName(), rightTriangle, ITERATIONS) 39 System.out.printf("Warm-up done ! \n\n"); 40 41 System.out.printf("Starting measurement intervals ... \n"); 42 doTest(Square.class.getSimpleName(), square, ITERATIONS); 43 doTest(Rectangle.class.getSimpleName(), rectangle, ITERATIONS); 44 //doTest(RightTriangle.class.getSimpleName(), rightTriangle, ITERATIONS) 45 System.out.printf("Measurement intervals done ! \n\n"); 46 } 47 48 private static void doTest(String message, Shape shape, long iterations) { 49 50 System.out.printf("Test [%s] \n", message); 51 52 long start = System.nanoTime(); 53 for (long i = 0; i < iterations; i++) { 54 // area = shape.area(); 55 } 56 long elapsed = System.nanoTime() - start; 57 58 double elapsedInSec = nanoToSecConverter(elapsed); 59 double rate = elapsed / iterations; 60 61 System.out.printf("\t\t[%s]: area = [%7.2f] \n", message, area); 62 System.out.printf("\t\t[%s]: elapsed time = [%7.4f] sec \n", message, elc 63 System.out.printf("\t\t[%s]: one iteration took = [%5.2f] ns \n", message 64 } 65 66 private static double nanoToSecConverter(long time) { 67 return (double) time / (1000 * 1000 * 1000); 68 } 69 } 70 71 abstract class Shape { 72 73 //----- 74 // Methods 75 //----- 76 77 abstract double area(); 78 } 79 80 }</td><td>0: getstatic #2 // Field java/lang/System.out:Ljava/io/PrintStream; 3: ldc #30 // String Test [%s] \n 5: iconst_1 6: anewarray 9: dup 10: iconst_0 11: aload_0 12: aastore 13: invokevirtual #7 // Method java/io/PrintStream.printf:(Ljava/lang/String:[16: pop 17: invokestatic #31 // Method java/lang/System.nanoTime:()J 20: lstore 22: iconst_0 23: lstore 25: lload 27: lload_2 28: lcmp 29: ifge 32: aload_1 33: invokevirtual #33 // Method java/lang/Shape.area:()D 36: putstatic #33 // Field area:D 39: lload 41: iconst_1 42: ladd 43: lstore 45: goto 48: invokestatic #31 // Method java/lang/System.nanoTime:()J 51: lload 53: lsub 54: lstore 56: lload 58: invokestatic #34 // Method nanoToSecConverter:(J)D 61: dstore 63: lload 65: lload_2 66: ddiv 67: lld 68: dstore 69: getstatic #2 // Field java/lang/System.out:Ljava/io/PrintStream; 73: ldc #35 // String \t\t[%s]: area = [%7.2f] \n 75: iconst_2 76: anewarray 79: dup 80: iconst_0 81: aload_0 82: aastore 83: dup 84: iconst_1 85: getstatic #33 // Field area:D 88: invokevirtual #36 // Method java/lang/Double.valueOf:(D)Ljava/lang/Double; 91: aastore 92: invokevirtual #7 // Method java/io/PrintStream.printf:(Ljava/lang/String:[95: pop</td><td>0x0000000002556048: int3 0x0000000002556049: mov #0xffffffff9,&edx 0x000000000255604e: mov %r14,%rbp 0x0000000002556051: rex 0x0000000002556056: rex 0x000000000255605b: mov %r13,0x28(%rbp) 0x0000000002556060: mov %rbx,0x30(%rbp) 0x0000000002556065: mov %r8,0x38(%rbp) 0x000000000255606a: nop 0x000000000255606b: callq 0x00000000025274e0 ; OopMap[rbp=Oop [56]=Oop off=1328] ; *load ; - JitShapeMain::doTest@25 (line 53) ; (runtime_call) 0x0000000002556070: int3 ; *load ; - JitShapeMain::doTest@25 (line 53) 0x0000000002556071: mov %r9,0x40(%rbp) 0x0000000002556076: jmpq 0x0000000002556055 0x000000000255607b: mov %rbx,%r11 0x000000000255607e: mov #0xffffffffc,&edx 0x0000000002556083: mov %r9,%rbp 0x0000000002556086: mov %r13,0x30(%rbp) 0x000000000255608b: mov %r11,0x38(%rbp) 0x0000000002556090: mov %r14,0x40(%rbp) 0x0000000002556095: xchq %rax,%rax 0x0000000002556097: callq 0x00000000025274e0 ; OopMap[rbp=Oop [64]=Oop off=1372] ; *invokevirtual area ; - JitShapeMain::doTest@30 (line 54) ; (runtime_call) 0x0000000002556098: int3 0x000000000255609d: mov #0xffffffff6,&edx 0x00000000025560a2: nop 0x00000000025560a3: callq 0x00000000025274e0 ; OopMap[off=1384] ; *invokevirtual area ; - JitShapeMain::doTest@33 (line 54) ; (runtime_call) 0x00000000025560a8: int3 ; *newarray ; - JitShapeMain::doTest@127 (line 63) 0x00000000025560a9: mov %rax,&rdx 0x00000000025560ad: jmp 0x00000000025560d9 ; *newarray ; - JitShapeMain::doTest@102 (line 62) 0x00000000025560ae: mov %rax,&rdx 0x00000000025560b1: jmp 0x00000000025560d9 ; *newarray ; - JitShapeMain::doTest@76 (line 61) 0x00000000025560b3: mov %rax,&rdx 0x00000000025560b6: jmp 0x00000000025560d9 ; *invokevirtual printf ; - JitShapeMain::doTest@142 (line 63) 0x00000000025560b8: mov %rax,&rdx 0x00000000025560bb: jmp 0x00000000025560d9 ; *invokestatic valueOf ; - JitShapeMain::doTest@138 (line 63) 0x00000000025560bd: mov %rax,&rdx 0x00000000025560c0: jmp 0x00000000025560d9 ; *invokevirtual printf ; - JitShapeMain::doTest@117 (line 62) 0x00000000025560c2: mov %rax,&rdx</td></tr></tbody></table></div><div data-bbox="34 947 101 965" data-label="Page-Footer">www.luxoft.com</div><div data-bbox="924 938 973 988" data-label="Page-Footer">LUXOFT</div>		

JITWatch - mega-morphic call

TriView - Source, Bytecode, Assembly Viewer - JITWatch

Class: JitShapeMain

Member: private static void doTest(String,Shape,long)

☒ Source ☒ Bytecode ☒ Assembly ☐ Chain ☐ Journal ☒ Mouse Follow

Bytecode size Native size Compile time [...]

147 1392 10

Source

```
27 // Run this with -XX:+UnlockDiagnosticVMOptions -XX:PrintAssembly and f
28
29 System.out.printf("Running application with Java version [%s] \n", System
30
31 Shape square = new Square(25.33);
32 Shape rectangle = new Rectangle(20.75, 30.25);
33 Shape rightTriangle = new RightTriangle(20.50, 30.25);
34
35 System.out.printf("Starting warm-up ... \n");
36 doTest(Square.class.getSimpleName(), square, ITERATIONS);
37 doTest(Rectangle.class.getSimpleName(), rectangle, ITERATIONS);
38 doTest(RightTriangle.class.getSimpleName(), rightTriangle, ITERATIONS);
39 System.out.printf("warm-up done ! \n\n");
40
41 System.out.printf("Starting measurement intervals ... \n");
42 doTest(Square.class.getSimpleName(), square, ITERATIONS);
43 doTest(Rectangle.class.getSimpleName(), rectangle, ITERATIONS);
44 doTest(RightTriangle.class.getSimpleName(), rightTriangle, ITERATIONS);
45 System.out.printf("Measurement intervals done ! \n\n");
46
47 }
48 private static void doTest(String message, Shape shape, long iterations) {
49
50     System.out.printf("Test [%s] \n", message);
51
52     long start = System.nanoTime();
53     for (long i = 0; i < iterations; i++) {
54         area = shape.area();
55     }
56     long elapsed = System.nanoTime() - start;
57
58     double elapsedInSec = nanoToSecConverter(elapsed);
59     double rate = elapsed / iterations;
60
61     System.out.printf("\t[%s]: area = [{},7.2f] \n", message, area);
62     System.out.printf("\t[%s]: elapsed time = [{},7.4f] sec \n", message, el
63     System.out.printf("\t[%s]: one iteration took = [{},5.2f] ns \n", message
64 }
65
66 private static double nanoToSecConverter(long time) {
67     return (double) time / (1000 * 1000 * 1000);
68 }
69
70
71 abstract class Shape {
72
73     //-----
74     // Methods
75     //-----
76
77     abstract double area();
78 }
79
```

Bytecode (double click for JVM spec)

```
0: getstatic #2 // Field java/lang/System.out:Ljava/io/PrintStream;
3: ldc #30 // String Test [%s] \n
5: iconst_1
6: anewarray #4 // class java/lang/Object
9: dup
10: iconst_0
11: aload_0
12: astore
13: invokevirtual #7 // Method java/io/PrintStream.printf:(Ljava/lang/String;[L
16: pop
17: invokevirtual #31 // Method java/lang/System.nanoTime:()J
20: lstore 4
22: iconst_0
23: lstore 6
25: lload 6
27: lload_2
28: lcmp
29: ifge 48
32: aload 1
33: invokevirtual #32 // Method java/lang/String.format:(Ljava/lang/String;[L
36: putstatic #33 // Field area:D
39: lload 6
41: iconst_1
42: ladd
43: lstore 6
45: goto 25
48: invokevirtual #31 // Method java/lang/System.nanoTime:()J
51: lload 4
53: lsub
54: lstore 6
56: lload 6
58: invokevirtual #34 // Method nanoToSecConverter:(J)D
61: dstore 8
63: lload 6
65: lload_2
66: ldiv
67: l2d
68: dstore 10
70: getstatic #2 // Field java/lang/System.out:Ljava/io/PrintStream;
73: ldc #35 // String \t[%s]: area = [{},7.2f] \n
75: iconst_2
76: anewarray #4 // class java/lang/Object
79: dup
80: iconst_0
81: aload_0
82: astore
83: dup
84: iconst_1
85: getstatic #33 // Field area:D
86: invokevirtual #36 // Method java/lang/Double.valueOf:(D)Ljava/lang/Double;
91: astore
92: invokevirtual #7 // Method java/io/PrintStream.printf:(Ljava/lang/String;[L
95: pop
```

Assembly ☐ Local labels

```
0x0000000002849875: mov %rdi,0x38(%rsp) ;*newarray
; - JitShapeMain::doTest@6 (line 50)
0x000000000284987a: lea (%r12,%rbp,8),%rdx ;*getstatic out
; - JitShapeMain::doTest@0 (line 50)
0x000000000284987e: movabs %0x7d5c5990,%rsi ; oop(%quot;Test [%s] quot;)
0x0000000002849888: data16 %chx %ax,%ax
0x000000000284988b: callq 0x0000000002817c60 ; OopMap[(32)=Oop [40]=Oop [56]=NarrowC
;*invokevirtual printf
; - JitShapeMain::doTest@13 (line 50)
; (optimized virtual_call)
0x0000000002849890: movabs %0x675a79c0,%r10
0x000000000284989a: callq ; - JitShapeMain::doTest@17 (line 52)
0x000000000284989d: mov %rax,0x40(%rsp)
0x00000000028498a2: mov 0x30(%rsp),%r10
0x00000000028498a7: test %r10,%r10
0x00000000028498aa: jle 0x00000000028498f6 ;*ifge
; - JitShapeMain::doTest@29 (line 53)
0x00000000028498ac: xor %ebp,%ebp
0x00000000028498ae: jmp 0x00000000028498b5
0x00000000028498b0: mov 0x30(%rsp),%r10 ;*aload_1
; - JitShapeMain::doTest@32 (line 54)
0x00000000028498b5: mov %r10,0x30(%rsp)
0x00000000028498ba: mov 0x28(%rsp),%rdx
0x00000000028498bf: %chx %ax,%ax
0x00000000028498c1: movabs %0xffffffffffffffff,%rax ; oop(NULL)
0x00000000028498cb: callq 0x0000000002817c60 ; OopMap[(32)=Oop [40]=Oop [56]=NarrowC
;*invokevirtual area
; (virtual_call)
; - JitShapeMain::doTest@35 (line 54)
0x00000000028498d0: movabs %0x7d5c5263,%r10 ; oop(a 'java/lang/Class' = 'JitShap
0x00000000028498da: vmovd %xmm0,0x78(%r10)
0x00000000028498de: lock addl %0x0,(%rsip) ;*putstatic area
; - JitShapeMain::doTest@36 (line 54)
0x00000000028498e5: add %0x1,%rbp ; OopMap[(32)=Oop [40]=Oop [56]=NarrowOop off=265)
;*goto
; - JitShapeMain::doTest@45 (line 53)
0x00000000028498e9: test %eax,-0x1ba98ef(%rip) # 0x000000000000ca000
;*goto
; - JitShapeMain::doTest@45 (line 53)
; {poll}
0x00000000028498ef: cmp 0x30(%rsp),%rbp
0x00000000028498f4: jl 0x00000000028498b0 ;*ifge
; - JitShapeMain::doTest@29 (line 53)
0x00000000028498f6: movabs %0x675a79c0,%r10
0x0000000002849900: callq ; - JitShapeMain::doTest@48 (line 56)
0x0000000002849903: mov %rax,%r10
0x0000000002849906: sub 0x40(%rsp),%r10 ;*lsub
; - JitShapeMain::doTest@53 (line 56)
0x000000000284990b: mov %r10,%rbp
0x000000000284990e: mov %r10,%rdx
0x0000000002849911: %chx %ax,%ax
0x0000000002849913: callq 0x0000000002818060 ; OopMap[(32)=Oop [56]=NarrowOop off=31
;*invokestatic nanoToSecConverter
```

Micro-Benchmarking advice

- **Avoid measuring in Interpreter while benchmarking**
- **Have appropriate # of warmup iterations to reach steady state**
 - Ideally several cycles of short warm ups
- **Be aware of Just-In-Time Compiler optimizations**
 - Virtual method calls optimizations
 - Inlining
 - On-Stack-Replacement

A collection of various blue geometric shapes including triangles, squares, and circles, some containing icons like a gear and a lightbulb, scattered on the left side of the slide.

THANK YOU

Ionuț Baloșin

Software Architect

ibalosin@luxoft.com

 [@ibalosin](https://twitter.com/ibalosin)



LXFT
LISTED
NYSE



Appendix

PrintCompilation

♦ 202 15 % 3 ! s b n com.luxoft.jpt.course.jit.fieldaccess.DfaCaller::call @ 16 (51 bytes)

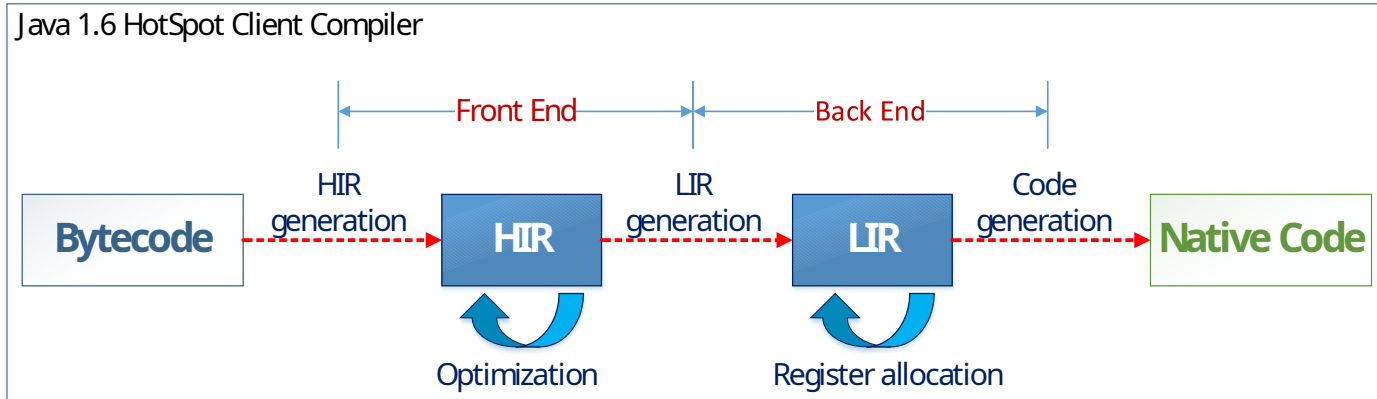
- ♦ 202 : millis from start JVM
- ♦ 15: sequence number of compilation
- ♦ %: On Stack Replacement
- ♦ 3: Compilation Level
- ♦ !: method has exception handlers
- ♦ s: synchronized method
- ♦ b: compilation occurred in blocking mode
- ♦ n: compilation occurred for a wrapper to a native method
- ♦ 16: On Stack Replacement bytecode index {osr_bci}
- ♦ 51 bytes: number of bytes contained in compiled method

Code Cache

- ♦ **Code Cache** – holds JVM compiled code (set of assembly-language instructions)
“hidden space” cleaned by JIT Compiler threads
- ♦ **Warnings:**
 - ♦ CodeCache is full. Compiler has been disabled
 - ♦ CodeCache is full. Try increasing the code cache size using `-XX:ReservedCodeCacheSize`

JVM arguments	Explanations
<code>-XX:InitialCodeCacheSize</code>	Initial code cache size (in bytes)
<code>-XX:ReservedCodeCacheSize</code>	Maximum/reserved code cache size to be used
<code>-XX:+UseCodeCacheFlushing</code>	Attempt to sweep the code cache before shutting off compiler. Default with Java 7u4

Front End / Back End Compiler



- ♦ **SSA** – Static Single Assignment
- ♦ **CFG** – Control Flow Graph
- ♦ **HIR** – High Level Intermediate Representation
- ♦ **LIR** – Low Level Intermediate Representation

- ♦ **HIR** optimizations:
 - method inlining
 - loop unrolling
 - null check elimination
- ♦ **LIR** register allocation:
 - maps virtual to physical registers

Just-In-Time Compiler optimizations

Loop unrolling – reduces cost of jump instructions that control the loop

```
for (x = 0; x < 100; x++) {  
    compute(x);  
}
```



```
for (x = 0; x < 100; x += 5) {  
    compute(x);  
    compute(x + 1);  
    compute(x + 2);  
    compute(x + 3);  
    compute(x + 4);  
}
```

Null check elimination – eliminates unreachable null code branch

```
private void enroll() {  
    JavaPerformanceAndTuning jtp = new JavaPerformanceAndTuning();  
    if (null == jtp)  
        throw new NullPointerException("Cannot enroll to null course");  
    jtp.enroll();  
}
```

Just-In-Time Compiler optimizations

Lock Eliding – when only the thread that created the lock will ever have access to it, can safely be omitted

```
public void foo() {  
    List list = new ArrayList();  
    synchronized (list) {  
        list.add(getElement());  
    }  
}
```



```
public void foo() {  
    List list = new ArrayList();  
    list.add(getElement());  
}
```

Lock coarsening – merges two critical sections, along with the non-critical section, into a larger block

```
synchronized (object) {  
    /* critical section */  
}
```

```
{  
    /* non-critical section */  
}
```

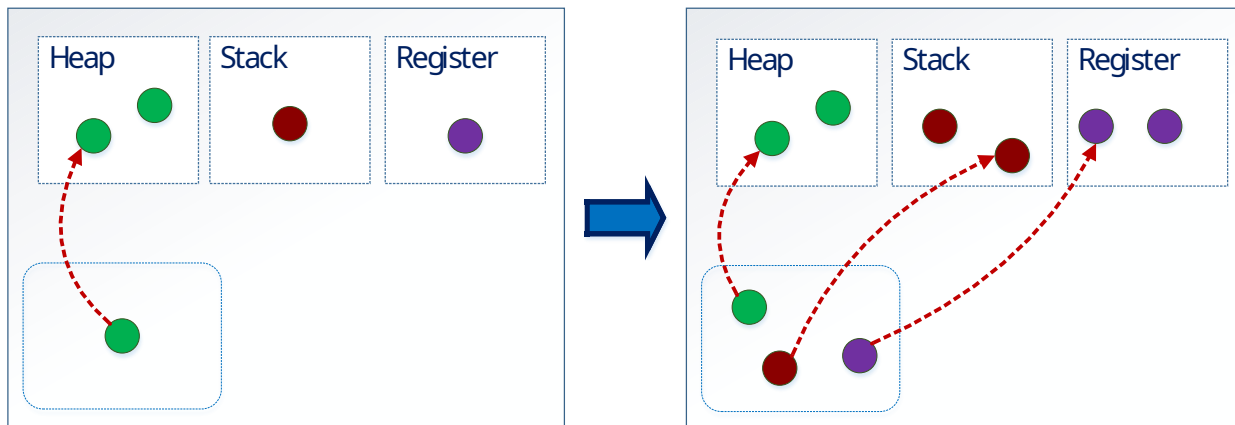
```
synchronized (object) {  
    /* critical section */  
}
```



```
synchronized (object) {  
    /* critical section */  
    /* non-critical section */  
    /* critical section */  
}
```

Just-In-Time Compiler optimizations

- **Escape Analysis** - analyze the scope of a new object's and decide whether to allocate it on the Java heap
- uses **scalar replacement** technique which removes object allocation by “object explosion” (prefer Register allocation, spills to Stack if necessary)



JVM arguments	Explanations
<code>-XX:+DoEscapeAnalysis</code>	Allocation optimization. Enabled by default in Java 6 u23